

Query processor or interface with database

Data :- Collection of raw fact or figure that can be processed to derive meaning or knowledge. In other term we can say any fact that can be stored Ex - Xyz, 12

Information :- Processed data is called information. Ex - game name, temperature.

Ex - [12] is raw fact it can be [12] - [Room No.]

*** Database :-**

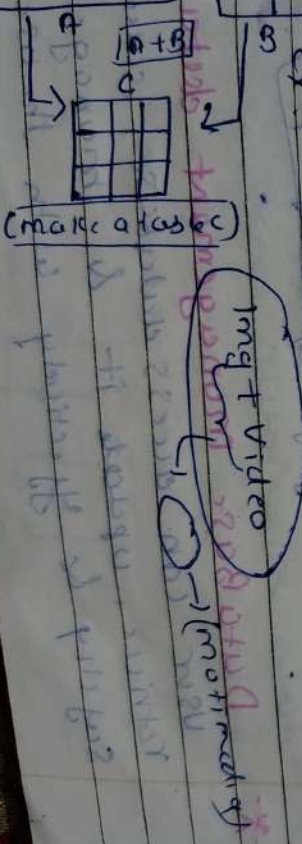
Collection of uncorrelated data.

It can be store in the form of table.

Ex :- multi media database

ID	nom	place
1	A	gog
2	B	gndor
3	C	gned

ID	Num	Sub
1	A	Ph6
2	B	CSO
3	C	ENG



img + video

(motion)

No sql :- For data in Non-relational format, such as key-value pairs, document, graph & column of design. To handle large-scale data & provide high performance for scenarios where relational model might be restrictive.

Object oriented database :- In object oriented programming, data is stored in object. To integrate object-oriented program concept into the database environment, allow data to store as obj. Support for complex data type & relationship make it ideal for applications requiring data modeling & real world simulation (object OR).

9n-memory db :- 9n-memory db is in memory. Data is stored in primary memory. Inset are stored in memory. No of times a different instance of that kind of data are there in this. **spatial database :-** geometric space. **multimedia db :-** collection of multimedia text, video, gif etc.

No sql :- For data in Non-relational format, such as key-value pairs, document, graph & column of design. To handle large-scale data & provide high performance for scenarios where relational model might be restrictive.

Object oriented database :- In object oriented programming, data is stored in object. To integrate object-oriented program concept into the database environment, allow data to store as obj. Support for complex data type & relationship make it ideal for applications requiring data modeling & real world simulation (object OR).

9n-memory db :- 9n-memory db is in memory. Data is stored in primary memory. Inset are stored in memory. No of times a different instance of that kind of data are there in this. **spatial database :-** geometric space. **multimedia db :-** collection of multimedia text, video, gif etc.

Columnar data base :- Columnar data is stored as column not in rows. XMI is data store in XML. **New sql :-** Bridge b/w the gap in relational & No-sql databases. **Blockchain db :-** data is stored in form of Block.

Relational :- These database structure data into organized table that have predefined connection b/w them. data manipulation & querying are performed using sql. **nosql :-** these db are created to handle data that doesn't fit neatly into the strict set of traditional relational database. ex-mongo db.

Object oriented db :- These db hold object (data & action) utilized in object-oriented programming. They work well for application with intricate data design, like simulation or multimedia s/w.

Columnar data base :- Columnar data is stored as column not in rows. XMI is data store in XML. **New sql :-** Bridge b/w the gap in relational & No-sql databases. **Blockchain db :-** data is stored in form of Block.

Relational :- These database structure data into organized table that have predefined connection b/w them. data manipulation & querying are performed using sql. **nosql :-** these db are created to handle data that doesn't fit neatly into the strict set of traditional relational database. ex-mongo db.

Object oriented db :- These db hold object (data & action) utilized in object-oriented programming. They work well for application with intricate data design, like simulation or multimedia s/w.

In memory DB :- These DB data is kept in the primary memory RAM rather than on a disk, allowing it to have data retrieval. They are employed in applications that demand instant data processing & top-notch performance. [Time of consuming data is very less]

* Needs :-

- effectively manage data.
- ensure data accuracy.
- Security.
- Supporting essential decision making process.

* Pros :-

① Data Security :- mechanism that regulate access to sensitive information. Safeguarding it from unauthorized access and potential data breaches.

② Data Redundancy & Inconsistency

DBMS ensure redundancy & ensure consistency through the maintenance of a unified version of a data.

③ Data Integrity :- Some rules & constraint so, any incorrect or inconsistent data not enter in database.

④ Scalability :- It handle large data set & scale accommodate increase amount of data as an organisational growth.

ex. if data 1-40 employ
New recruitment 1200+ employee hire

DB is easily 1 can hold 9151572

1-1240

⑤ Data Abstraction :- DBMS offer data abstraction allowing user & applications to interact with the database without needing to understand it's underlying complexities.

Cons:-

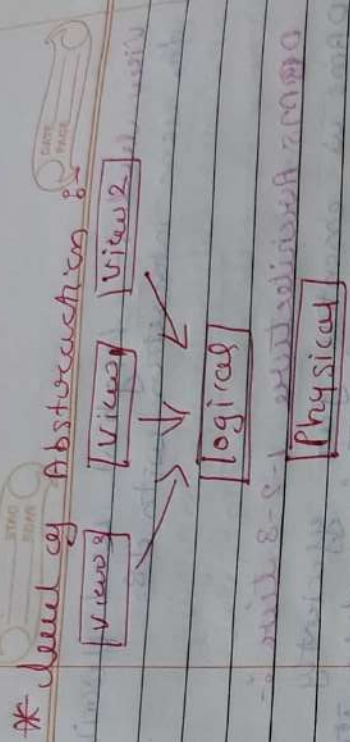
- 1) Cost is high
- 2) Scale project (I have to store only info of 20 but database is of 100)
- 3) Vendor lock-in :-
We currently use MySQL & Java (which have some set of vendor).
After some time I want to switch to MongoDB (I can't do this)

SQL — difficult
(To store data)
→ Rule diff of SQL & NoSQL

* Data Abstraction :-

(Collection name) (indexing)
fact

• hierarchy of unnecessary thing from user is called data abstraction.
like when it store, what technique it use, it's indexing & data structure plus (hide all this)



→ physical level :- lowest level it describe how data is actually store in DB. you can get the complete data structure details at this level.

→ Logical level :- middle level of 3 level archt. describe about data is store in database.
EX:- Student Database
Students

creation, addition attribute, & template & provide unique Id etc. (column)

View level :- highest level possible the user interaction with DB.

DBMS Architecture 1-2-3 tier :-

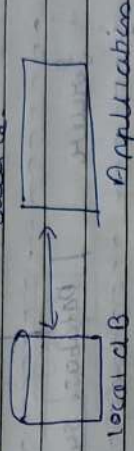
DBMS is essentially built efficiently storing, managing & retrieving data. Its architecture determines how data is organized, accessed & maintained choosing the right DBMS architecture ensure quick & secure & reliable data access.

Types :-

1-Tier Architecture :-
client, server & database all are present in same machine. Setup is simple & is often used in personal or standalone appls. where user interact directly with database.
ex :- Microsoft Excel Spreadsheet
is a great example of one-tier architecture.

Everything User interface, appl logic & data handle in single system.

User directly interact with application perform computation like calculation for data entry & store data locally on same machine.



2-Tier Architecture :-

is a fundamental client server model where the client application communicate directly with the DB. commonly used in scenarios where simplicity & security interaction b/w clients & servers are desired.

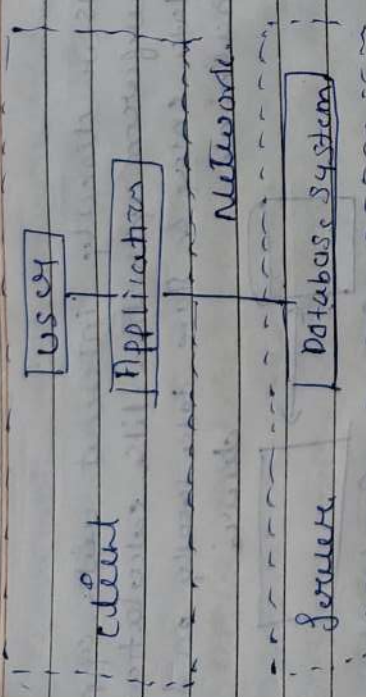
In this communication is facilitated through application programming interface like (API) :-

ODBC :- (open database connectivity)

Standard for API for accessing DBMS.

JDBC :- Java based API that enable

Java application to interact with DB.



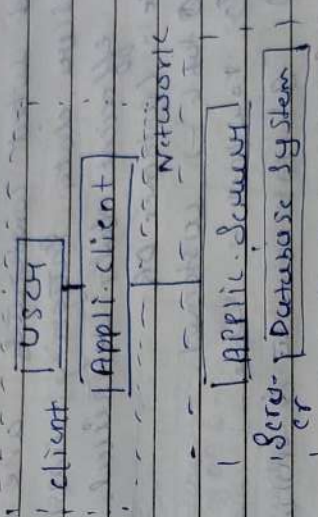
→ Easy to Access, low cost, Simple

3 Tier Architecture :-

There is another layer b/w the client & the server. The client not directly communicate with server. Instead it interact with an appli. server which further communicate with db system. & query processing & transaction manage take place. Indirectly they act as a medium for exchanging data b/w client & server. Server of particular process data b/w client & server.

ex - e-commerce store.
User -> You visit online store, search for a product & add it to your cart

Processing -> System check if the product in stock calculate T.P & apply discount. database -> details, your cart & order history are store in the database for future use (references).



Architecture -> it defines how data is stored manage & accessed in db system. It organise the database into layer or tiers to ensure efficient data processing, better scalability & easier management.

ex - e-commerce store

User -> You visit online store, search for a product & add it to your cart

SCHEMA AND INSTANCE :-

A logical container or structure that organizes & defines the structure of a database.

Blueprint of database that defines how data is organized, what data types are used, what constraints are applied, ensure data integrity, consistency & efficient data retrieval.

like - Customer DB.

- ① table - customer
- ② row/col - table
- ③ give column value & constraint.

Types :-

① physical schema :- defines how data is stored on online how it is organized. Such as storage, format, file organization, indexing method & data placement.

where my data store on the ^{disk} ~~file~~

Characteristics :-

- Store in proper (which help to retrieve & boost performance).
- modification made to the physical

Schema design & implementation can potentially affect overall performance. ex:- decide to use clustered indexes on specific col for faster retrieval.

② Logical Schema :- define lowest data in store.

Logical Schema

Conceptual

overall view of DB & high level structure & relationship between elements

External View

define user specific view of DB. It focus on portion of DB that are relevant to specific user roles or applic.

there are 30 accounts user specification view show

view show

view show

view show



Characteristics Logical Schema
 → how data is structure into tables & restriction placed on the data & interconnection b/w tables.

→ provide data modeling & database design with consideration related to hardware or storage specifics.

ex:- defining tables, specifying primary key & foreign key & creating view of database

Instance:- The info residing within a database at a specific point in time is referred to as the "instance".

→ "Instance" In this context denotes the current value of those variable at a particular moment in the time for that database.

Customer (table)

Id	Order	Address	2	Id	Order	Id
1	2	Bangalore	week	1	2	1
2	5	MS	(Gtr)	2	5	2

change MS - Pune

DBMS Architecture

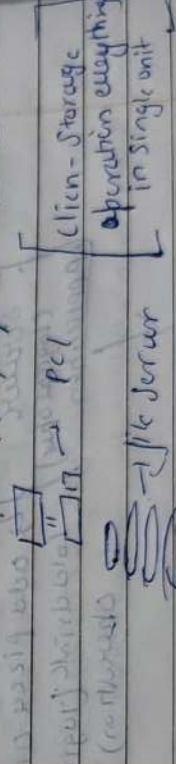
Refer to structural framework & organization of database management system. It defines various components of system work together to store, manage & retrieve data efficiently.

[It help to achieve a way through which we can easily organize, retrieve our database system]

Type 1 - Tier Architecture

entire database applications include user interface, application logic & data storage, reside on a single machine or computer.

ex:- ? Install database lang appli like Sql on my system I can perform any operation like create, delete, update etc.

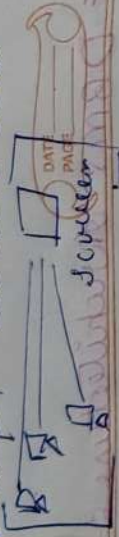


Type 2 - Tier Architecture

Two entity 1st client and server client making request to server & server making some manipulation & processing & give response to server. Client (2000)



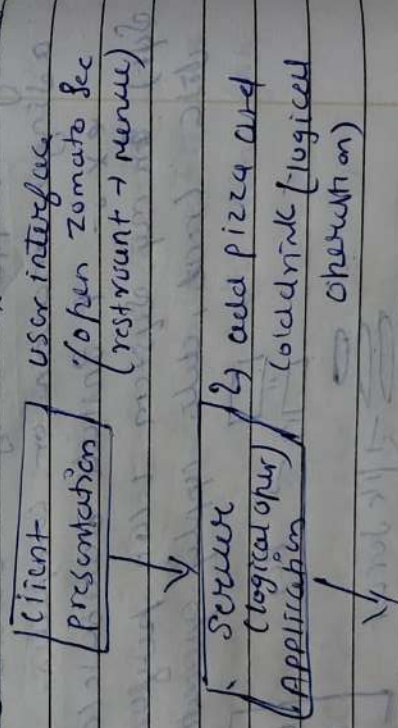
multiple comp → use same server



Client (User) → Server

- 3. Three - Separation of application into 3 logical distinct layers
- (1) presentation layer → handles interface
[user interface] → [user (client)]

- (2) Application layer → Manage some logic
[presentation] → [server] → [data layer]
- (3) data layer → storage & processing of database.
ex: Order on Zomato



- make change → scalable & flexible
(most use) Secure

Pros & Cons

- Scalability → each line is separate
So don't affect other.
- modularity & maintainability → easy
- Security → prevent additional layer.
- performance → optimize every layer acc. to require. for performance.

Cons

- Increase Complexity.
- potential latency issue.
- longer development time

* Data Model

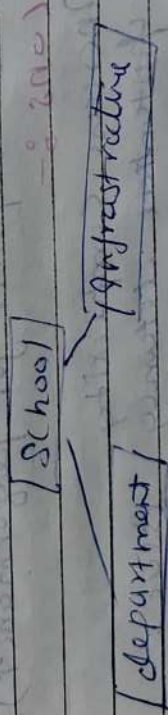
concept of schema (high level view) of db. It define an abstract representation of how data is structured & organized within a database.
It contains logical connection of data & connection b/w various data components.

Use a flow chart (like) to make a maggle (1) Ball water (2) Put maggie in water (3) Boil. So, it's a flow chart through which we can create maggie.

* Type :-

1. Hierarchical data model :-

- It resembles data in a tree like structure.
- record maintain as parent-child or every child has 1 parent.
- older database (use in XML).

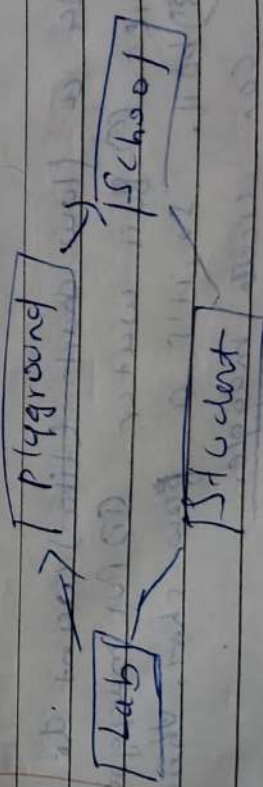


Teacher

Staff

2. Network DB :-

Similar to hierarchical approach, permitting record to hold multiple parent-child relationship. adopt a structure of graph. more flexible compare to hierarchy.



Lab

Playground

School

Student

• Relational data model :-

Organise data into tables (known as relations) consisting of rows & columns characterised the relational model. It stands as the most prevalent data model, rooted in the principle set theory & relies on (SQL) for data manipulation.

[There are many table and all those table have some relation b/w them that's why it's R.D.M.]

IP	Name	Place
1	Pahar	dahaj
2	Ref	Kolkata
3	Rit	Mumbai

Primary key

→ Row/Tuple

↓ column / Attribute

- E-R (model) :- (deal with entity (obj) and their relation) use for crafting relational database. ER model represent data through entity (object), attribute (entity properties) & relationship connecting these entities.

It's a high level data model which is used to define data & the relationship b/w them. It's basically a conceptual design of any database which is easy to design the views of data.

→ Component

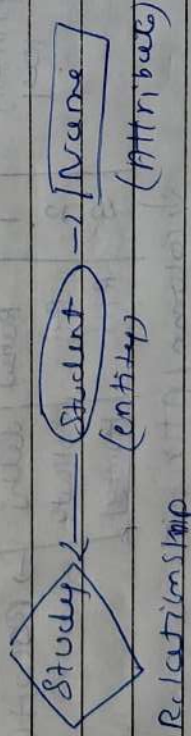
→ Entity → attribute → relationship

real world obj → obj (description) (realation b/w them)

Name, city, classmate → age, Roll No.

Symbol
Entity → marks

Ellipses →



It's a Conceptual D.M. oriented

Object V data model :- [Combination of ER & OOP]
Extend principle of OOP's in Database domain, this model depicts/describes data as objects complete with attribute & method fostering support for inheritance & encapsulation.

NOSQL DB :- NoSQL DB encompass a diverse variety of data model, such as document-oriented (eg. mongoDB) (key-value (eg. Redis), column-family (eg. Cassandra), and graph (eg. Neo4j). These models are designed to offer scalability & flexibility when handling extensive volume of unstructured & semi-structured data.

~~Data Independence~~ :- [any change that don't effect others] (virtual table or view based on user need)

View defined

Logical data Ind.

(any change done logical not effect view)

Physical data

Independence

(It any change in physical data don't effect logical)

Type

physical → logical data Ind. → any modification

(Here addition, change constraint doesn't effect other)

Company DB
A list of staff
table & employee
Staff manager
at error
Table view

clothes table

Baby's log

Used Baby's location

location

location - change

not affect

essential component of tables:-

→ Row (tuple) ⇒ Rows comprises Record/Tuple
represent individual entries or instance of data within the table

→ Cardinality → No. of row in table.
[Cardinality = 4]

table Book

1
2
3
4

→ Column/Attribute → col represent the attribute represent data being store and are name to describe the information they hold (eg "ID", "Name", "Age").

name	484342
------	--------

→ P.No.

→ degree → No. of col in table.

* essential component of Tables:-

Constraints :- set of rule or condition that must be satisfy by data in table.

ex:- P.No. enter in form must be 10 digit

P.No. = 10 digit
input must be 10 digit

Common constraints include uniqueness, nullability, default value etc.

Constraints :-

→ Unique constraint → ensure value in a column are unique across the table.

ex ⇒ Id in student table must unique

52	name	age
----	------	-----

→ Not Null → ensure column can't have Null value
→ Check constraint → enforce condition to be true for each row.

ex:- You making vector in the age
→ default → provide default value for a column if no value is specified

* If column is primary key then it must be unique and not null

Key in DBMS make source of data integrity & quick retrieval of info. Key is attribute

* [KEYS]

Key is one of the basic requirement of the relational model. We also use key to set up relation among various col & table of a relational database.

• **PRIMARY KEY** :- Uniquely identify each record in the table. It ensure the row can be uniquely identify and accessed within the table.

Id	Name	Add	PINO
1	Janu	Pune	11128
2	Janu	Pune	11128

It can't be NULL as it need to provide a valid, unique identification for every record.

FOREIGN KEY :-

F.K is a field in a table then refer to the primary key of another table. It establish relationship b/w tables. essential for maintaining data integrity & preventing data redundancy. Referencing table $\rightarrow$ table having PK $\rightarrow$ having (foreign key)

Candidate Key = {SID, SPHONE, ADDRESS}
PK = {SID} $\rightarrow$ Not Null & Unique

Base table

Rolling	Name	Address
1	A	delhi
2	B	Pune
3	A	US
4	D	Bangalore

Referencing table

Rolling	CIN	Roll
1	P888	1
2	CN	2
3	CS	3

• SUPER KEY :-

Set of one or more column that can uniquely identify a tuple in know SK.
It can contain extra attribute that aren't necessary for uniqueness exist if "STUD-NO" column can uniquely identify a student, adding "SNAME" to it will form a valid Super Key.
ex $\rightarrow$ Student table

PK	STUD-NO	SNAME	Address	PINO
1	1	RAM	delhi	11128
2	2	SHIVI	115	242312
3	3	SHIVAM	Pune	4332844

$\rightarrow$ Super Key could be combination of STUD-NO & PHONE as this combination uniquely identify a Student.

Combination of set of attribute which can identify a tuple

CANDIDATE KEY :-

minimised set of attributes which uniquely identify a tuple is known as a candidate key.
 → every table must have single candidate key
 → A candidate key must contain unique value, ensure no 2 rows have the same value in the candidate key.

ex 2:- For student table StudNo can be a candidate key, uniquely identify it.

Stud-No	Name	add	PH
1	RAH	X	124
2	SHIV	Y	328
3	SUNNY	Z	449

Table Stud-course

STUNo	TeachNo	Cours Id
1	001	(001)
2	002	(002)
3	020	(003)

{ StudNo, CourseNo } can be a candidate key for a student-course table.

COMPOSITE KEY :-

→ It act as a P.K if there is no P.K.
 → Two or more attributes are used together to make a composite key.
 ex:- Student-course table, Stud-No, CourseNo can form a composite key to avoid identify each row.

candidate key
 foreign key

empID	empName	empLoc	Plat	clid	clid design
001	tom	EL101	PA124	2	BPO
002	dohm	EL202	PI22A	3	ALC
003	Alice	EL303	PA211	5	IT

primary
 alternate key

unique key

PK and Alt

Alt-key :- All the key that are not selected as a primary key.